SIMPLE RING COUNTER

ENG2020 - DIGITAL ELECTRONICS 2

Kartike Sharan (3008620s)
Lecturer: Dr Scott Roy
December 2025

Contents

1 Aims and Objectives **3**

2 Background Theory **3**
2.1 Synchronous Systems and Shift Registers 3
2.2 Ring Counters 4
2.3 Experimental Hardware and Software 4

3 Methodology and Circuit Setup **5**
3.1 Block Diagram Construction 5
3.2 Waveform Simulation Setup 6

4 Hardware Verification **6**

5 Discussion **7**

6 Conclusion **7**

7 References **8**

1 Aims and Objectives

The aim of this lab was to design and test an 8-bit Simple Ring Counter using the Intel Quartus Prime environment and the MAX 10 FPGA board. The idea was to take an existing 8-bit shift register *(Shift_Register_8bit.vhd)* and convert it into a ring counter by feeding the last output bit (Q7) back into the serial input.
To complete this, the following steps were carried out:

- **Design and Implementation:** Build the circuit in Quartus and add the feedback connection so that the bit pattern continuously circulates.
- **Simulation:** Load the initial value `00000001` using the parallel load inputs and check through simulation that the pattern repeats every eight clock cycles.
- **Hardware Testing:** Program the FPGA and test the counter using both the push-button clock and an external TTL clock source to confirm that it behaves consistently under different timing conditions.

2 Background Theory

2.1 Synchronous Systems and Shift Registers

In this lab we worked with synchronous digital systems, meaning all of the important changes in the circuit happen in step with the clock signal. Every flip-flop updates on the rising edge, which keeps the behaviour predictable and avoids timing mismatches.

The main component we used was an 8-bit shift register. It consists of eight D-type flip-flops connected in a chain, where each clock pulse shifts the stored bits one position along the register. The version provided for the lab (Shift_Register_8bit.vhd) supports both normal shifting and an asynchronous parallel load. The parallel load was especially useful because it allowed us to set the starting pattern directly before the ring counter began circulating it.

Shift_Register_8bit.vhd

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity Shift_Register_8bit is
6    port(Clock,loadBr : in std_logic;
7         D : in std_logic_vector(7 downto 0);
8         SerIn : in std_logic;
9         Q : out std_logic_vector(7 downto 0));
10 end Shift_Register_8bit;
11
12 architecture behaviour of Shift_Register_8bit is
13 signal Regs : std_logic_vector(7 downto 0);
14 begin
15   process(Clock,D,LoadBr,SerIn,regs)
16   begin
17     if LoadBr = '0' then
18       Regs <= D;  -- asynchronous Load
19     else
20       if rising_edge(clock) then
21       Regs <= Regs(6 downto 0)& SerIn;
22       end if;
23     end if;
24   end process;
25   Q <= regs;
26 end behaviour;
```

Figure 1: VHDL for an 8-bit Shift Register.

2.2 Ring Counters

A ring counter is basically a shift register where the last output bit is fed back into the serial input. This creates a loop so that whatever pattern is inside the register keeps rotating around each clock cycle. In our case, we used an 8-bit shift register, so once the initial pattern (for example, 00000001) is loaded, it moves through each flip-flop and returns to the start after eight clock pulses.
This feedback connection is what turns a normal shift register into a ring counter, and it makes the behaviour easy to visualise on the FPGA since the single '1' appears to move along the LEDs in a repeating cycle.

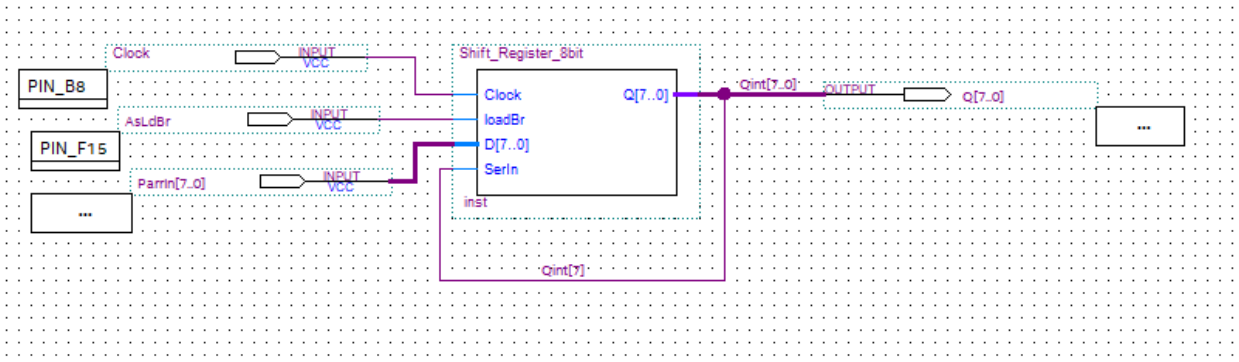

Figure 2: Block Diagram for a Ring Counter.

2.3 Experimental Hardware and Software

- **MAX 10 DE10-Lite FPGA board:** For this lab, we used the MAX 10 DE10-Lite FPGA board. Most of the inputs and outputs were taken from the built-in switches and LEDs, which made it easy to see the state of the ring counter while it was running. The slide switches `SW[7:0]` were used for the parallel input, `SW[8]` acted as the load signal, and the LED array `LED[7:0]` showed the contents of the register in real time. The push button `KEY[0]` was used whenever a manual clock pulse was needed.

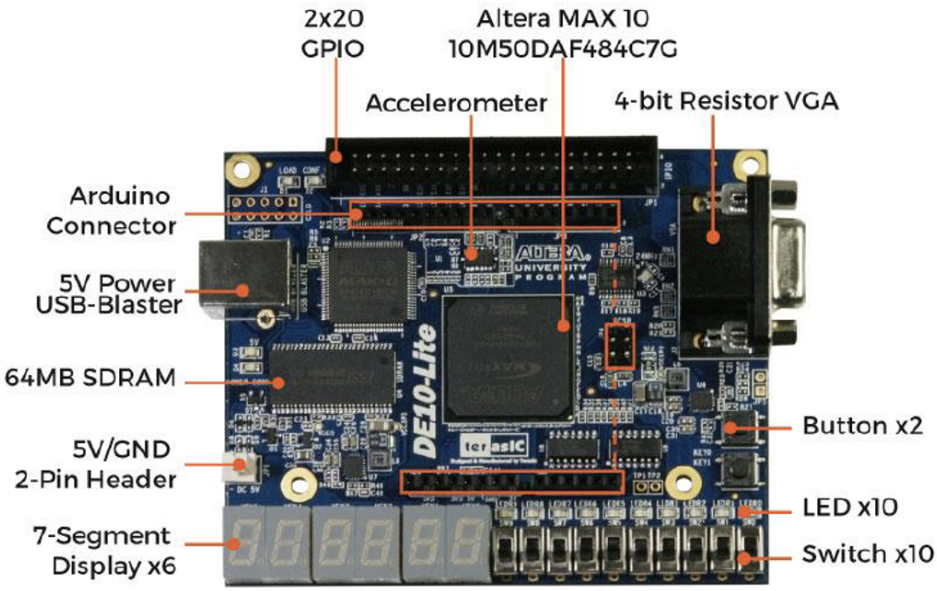

Figure 3: Block diagram of the DE10-Lite FPGA used in the experiment.

- **Intel Quartus Prime Software:** All of the design work was done in Intel Quartus Prime. This included the VHDL code, drawing the block diagram, running the simulations, and setting the pin assignments before programming the FPGA.

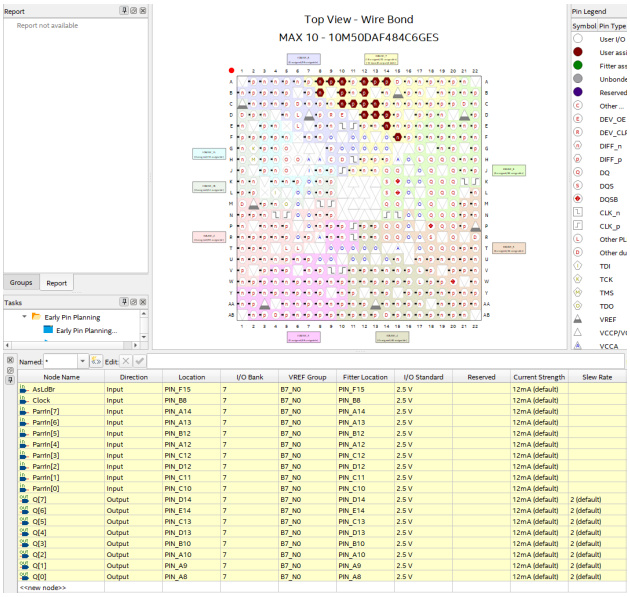

Figure 4: Pin assignment view used in Quartus Prime.

- **Signal Generator (GW Instek GFG-8216A):** This external device provided a stable TTL clock signal. It was connected to the BNC clock input (pin `AB5`) on the FPGA to test the counter under higher-frequency conditions.
- **Digital Storage Oscilloscope (Tektronix DPO2002B):** The oscilloscope was used to monitor the external clock signal and verify that its edges were clean before connecting it to the FPGA.

3 Methodology and Circuit Setup

3.1 Block Diagram Construction

The design relies on a chain of eight cascaded D-type flip-flops, implementing the Serial-In Parallel-Out (SIPO) architecture. The input and output signals define the external interface of the register.

- `CLK`: Clock signal provides the timing reference for synchronous data shifting.
- `RST`: Asynchronous reset signal is used to clear the register immediately.
- `SER_IN`: Single input for serial data feeding into the least significant bit (Q[0]) from the most significant bit (Q[7]).
- `Parrin(7 downto 0)`: 8-bit input providing all register contents simultaneously.
- `Q(7 downto 0)`: 8-bit output providing all register contents simultaneously to the LEDS on the FPGA.

Refer to figure 2

3.2 Waveform Simulation Setup

Before testing the design on the FPGA, I ran a functional simulation in Quartus to make sure the shift register and feedback behaved correctly. A simple VHDL testbench was used to generate the clock signal and apply a few test inputs.
The simulation mainly focused on two things:

1. **Asynchronous Reset:** I briefly set `RST` high to check that all outputs instantly cleared to `00000000`. This confirmed that the reset behaviour worked as expected.

2. **Serial Shifting:** After the reset, I applied a known bit pattern on `SER_IN` (`00000001`) and let it shift through the register for eight clock cycles. This helped confirm that each bit moved along correctly and appeared on the right output at the right time.

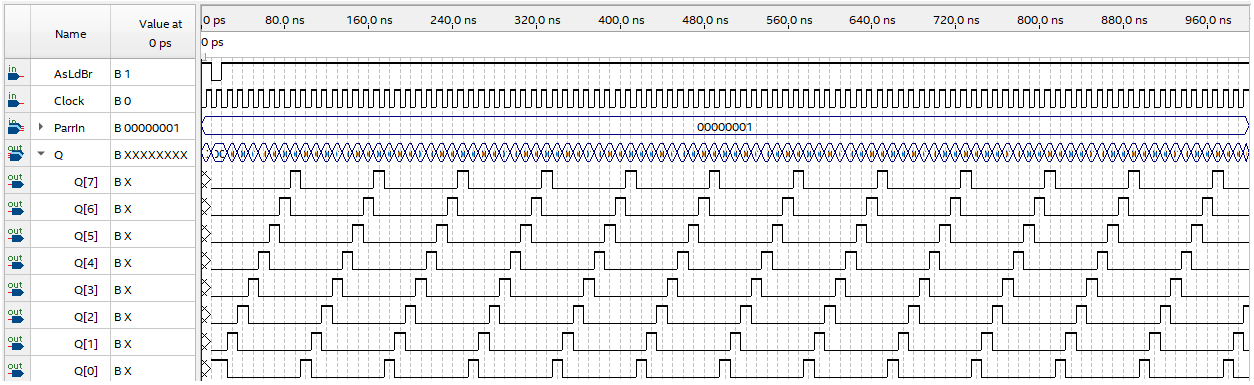

Figure 5: Simulated Waveform.

4 Hardware Verification

After uploading the planned pin assignments (Figure 4), the FPGA was connected to the waveform generator and the oscilloscope to help visualise the clocked bit transitions.

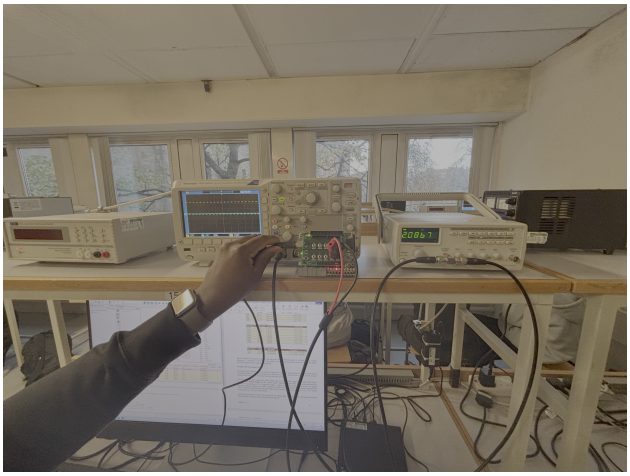

Figure 6: FPGA in between the oscilloscope and the waveform generator

5 Discussion

The 8-bit Simple Ring Counter was successfully implemented and verified, meeting all of the original design goals and confirming that the sequential circuit is both functionally correct and stable. The key technical takeaway is the crucial role of the asynchronous parallel load inputs (LdBr and ParrIn), which are needed to load the initial seed pattern 00000001; this step is essential to start the sequence and cannot be done using only the synchronous shift path. The design used very little hardware—just eight D-type flip-flops—and showed strong synchronous stability over a wide range of clock speeds, as seen in simulation and in hardware tests using both a manual push-button clock and an external signal generator.

6 Conclusion

The project's goal, to design, simulate, and verify a synchronous 8-bit Simple Ring Counter, was fully achieved. The circuit reliably loaded the initial pattern and produced a stable, repeating sequence with a period of exactly eight clock cycles. The combined verification approach, covering the VHDL implementation, functional simulation, and deployment on the MAX 10 FPGA with both push-button and external clock sources, shows that the design is a sturdy, low-resource sequence generator that is well suited for use in larger digital systems.

7 References

Roy, S. (2025). *ENG2020: Digital Electronics 2 - Simple Ring Counter Lab Sheet.* University of Glasgow, School of Engineering. (Course Materials).